

Guide de test - JobbingTrack

Vue d'ensemble

Ce guide détaille la stratégie de test complète de JobbingTrack, incluant les tests unitaires, d'intégration, E2E, de sécurité et d'accessibilité.

Approche recommandée : Utilisation des commandes Make

JobbingTrack utilise un système de **commandes Make** centralisées qui orchestrent tous les types de tests. Cette approche est **recommandée** car elle :

-  **Centralise** la logique de test
-  **Gère** les dépendances automatiquement
-  **Configure** l'environnement (Docker, DB, etc.)
-  **Produit** des rapports cohérents
-  **Évite** les erreurs de configuration

Commandes Make principales

```
# Tests complets (recommandé pour CI/CD)
make test-all

# Tests rapides (développement quotidien)
make test-quick

# Tests backend uniquement
make test-backend-only

# Tests frontend uniquement
make test-frontend-only

# Tests d'intégration
make test-integration

# Tests E2E
make test-e2e

# Tests de performance et sécurité
make test-performance
make test-security
```

```
# Tests avec coverage  
make test-coverage
```

Alternative : Commandes npm

Si les commandes Make ne sont pas disponibles ou pour un contrôle plus fin, utilisez les commandes npm :

```
# Commandes npm équivalentes  
npm run test:*      # Tests spécifiques  
node tests/*.js     # Scripts de test direct
```

Note : Les exemples ci-dessous utilisent d'abord les commandes Make quand disponibles, puis les alternatives npm.

Stratégie de test

Niveaux de test

Tests End-to-End (Playwright, Cypress)
Tests d'intégration (API, Services, Database)
Tests unitaires (Jest, React Testing Library)
Tests de composants/fonctions (Isolated testing)

Tests unitaires

Backend

Framework : Jest + Supertest

Commandes principales :

```
# Tests backend uniquement (recommandé)
make test-backend-only

# Tests unitaires backend
make test-unit

# Avec coverage
make test-coverage

# Alternative si make non disponible :
npm run test
npm run test:ci
npm run test:watch
```

Structure des tests :

```
backend/[service]/tests/
├─ setup.js           # Configuration Jest
├─ [feature].test.js  # Tests unitaires
├─ integration/       # Tests d'intégration
└─ fixtures/         # Données de test
```

Exemple de test :

```
const request = require('supertest');
const app = require('../src/server');

describe('API Tests', () => {
  test('GET /health should return 200', async () => {
    const response = await request(app)
      .get('/health')
      .expect(200);

    expect(response.body).toHaveProperty('status', 'ok');
  });

  test('POST /api/users should create user', async () => {
    const userData = {
      email: 'test@example.com',
      password: 'password123'
    };

    const response = await request(app)
      .post('/api/users')
      .send(userData)
      .expect(201);
```

```

    expect(response.body).toHaveProperty('id');
    expect(response.body.email).toBe(userData.email);
  });
});

```

Frontend

Framework : Jest + React Testing Library

Commandes principales :

```

# Tests frontend uniquement (recommandé)
make test-frontend-only

# Tests unitaires frontend
make test-frontend

# Tests avec coverage
make test-coverage

# Alternative si make non disponible :
npm run test
npm run test:ci
npm run test:unit

```

Exemple de test React :

```

import { render, screen, fireEvent } from '@testing-library/react';
import LoginForm from '../components/LoginForm';

describe('LoginForm', () => {
  test('renders login form', () => {
    render(<LoginForm />);

    expect(screen.getByLabelText('Email')).toBeInTheDocument();
    expect(screen.getByLabelText('Password')).toBeInTheDocument();
    expect(screen.getByRole('button', { name: 'Login' })).toBeInTheDocument();
  });

  test('handles form submission', async () => {
    const onSubmit = jest.fn();
    render(<LoginForm onSubmit={onSubmit} />);

    fireEvent.change(screen.getByLabelText('Email'), {
      target: { value: 'test@example.com' }
    });
  });
});

```

```
});
fireEvent.change(screen.getByLabelText('Password'), {
  target: { value: 'password123' }
});

fireEvent.click(screen.getByRole('button', { name: 'Login' }));

expect(onSubmit).toHaveBeenCalledWith({
  email: 'test@example.com',
  password: 'password123'
});
});
});
```

Tests d'intégration

Backend

Objectif : Tester les interactions entre services.

Commandes principales :

```
# Tests d'intégration (recommandé)
make test-integration

# Tests API complets
make test-api

# Tests de base de données
make test-database

# Alternative si make non disponible :
cd tests && npm run test:integration
node tests/api/test-api.js
node tests/database/test-database.js
```

Exemple :

```
describe('Auth Integration', () => {
  test('should authenticate and access protected route', async () => {
    // 1. Créer un utilisateur
    const userResponse = await request(app)
      .post('/api/auth/register')
      .send({
        email: 'test@example.com',
```

```

        password: 'password123'
    });

    // 2. Se connecter
    const loginResponse = await request(app)
        .post('/api/auth/login')
        .send({
            email: 'test@example.com',
            password: 'password123'
        });

    const token = loginResponse.body.token;

    // 3. Accéder à une route protégée
    await request(app)
        .get('/api/users/profile')
        .set('Authorization', `Bearer ${token}`)
        .expect(200);
    });
});

```

Frontend

Exemple avec React Query :

```

import { renderHook, waitFor } from '@testing-library/react';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { useAuth } from '../hooks/useAuth';

const createWrapper = () => {
    const queryClient = new QueryClient({
        defaultOptions: {
            queries: { retry: false },
            mutations: { retry: false },
        },
    });
};

return ({ children }) => (
    <QueryClientProvider client={queryClient}>
        {children}
    </QueryClientProvider>
);
};

describe('useAuth', () => {
    test('should login successfully', async () => {
        const wrapper = createWrapper();
        const { result } = renderHook(() => useAuth(), { wrapper });
    });
});

```

```

result.current.login({
  email: 'test@example.com',
  password: 'password123'
});

await waitFor(() => {
  expect(result.current.user).toBeTruthy();
  expect(result.current.isAuthenticated).toBe(true);
});
});
});

```

Tests End-to-End (E2E)

Configuration Playwright

Commandes principales :

```

# Tests E2E (recommandé)
make test-e2e

# Tests E2E avec interface graphique
make test-e2e-ui

# Tests avec interface
make test-e2e-ui

# Tests en mode debug
make test-e2e-debug

# Alternative si make non disponible :
npm run test:e2e
npm run test:e2e:ui
npm run test:e2e:debug

```

Structure :

```

frontend/tests/e2e/
├─ specs/
│   ├─ login.spec.ts
│   ├─ dashboard.spec.ts
│   └─ admin.spec.ts
├─ fixtures/
│   └─ test-data.ts

```

```
└─ utils/
  └─ test-helpers.ts
└─ playwright.config.ts
```

Exemple de test E2E :

```
import { test, expect } from '@playwright/test';

test.describe('Authentication', () => {
  test('should login successfully', async ({ page }) => {
    await page.goto('/login');

    // Remplir le formulaire
    await page.fill('[data-testid="email-input"]', 'test@example.com');
    await page.fill('[data-testid="password-input"]', 'password123');

    // Soumettre
    await page.click('[data-testid="login-button"]');

    // Vérifier la redirection
    await expect(page).toHaveURL('/dashboard');

    // Vérifier la présence du tableau de bord
    await expect(page.locator('h1')).toContainText('Dashboard');
  });

  test('should show error for invalid credentials', async ({ page }) => {
    await page.goto('/login');

    await page.fill('[data-testid="email-input"]', 'invalid@example.com');
    await page.fill('[data-testid="password-input"]', 'wrongpassword');

    await page.click('[data-testid="login-button"]');

    await expect(page.locator('[data-testid="error-message"]'))
      .toContainText('Invalid credentials');
  });
});
```

Tests multi-navigateurs

```
test.describe('Cross-browser testing', () => {
  // Tests exécutés sur tous les navigateurs configurés
  test('should work on all browsers', async ({ page }) => {
    await page.goto('/dashboard');
    await expect(page.locator('h1')).toBeVisible();
  });
});
```



```
});
});
```

Tests mobiles

```
test.describe('Mobile testing', () => {
  test.use({
    viewport: { width: 375, height: 667 }
  });

  test('should work on mobile', async ({ page }) => {
    await page.goto('/dashboard');

    // Tester le menu mobile
    await page.click('[data-testid="mobile-menu-button"]');
    await expect(page.locator('[data-testid="mobile-menu"]')).toBeVisible();

    // Tester le swipe
    await page.touchscreen.tap(100, 100);
  });
});
```

Tests d'accessibilité

Commandes principales :

```
# Tests d'accessibilité (recommandé)
make test-a11y

# Tests d'accessibilité avec axe-core
npm run test:a11y:axe

# Tests d'accessibilité complets
npm run test:a11y:all

# Alternative si make non disponible :
npm run test:a11y
```

axe-core automatisé

```
import AxeBuilder from '@axe-core/playwright';

test('should pass accessibility tests', async ({ page }) => {
  await page.goto('/dashboard');

  const accessibilityScanResults = await new AxeBuilder({ page }).analyze();

  expect(accessibilityScanResults.violations).toEqual([]);
});
```

Tests manuels

```
test.describe('Manual accessibility tests', () => {
  test('should be keyboard navigable', async ({ page }) => {
    await page.goto('/dashboard');

    // Test de la navigation au clavier
    await page.keyboard.press('Tab');
    await expect(page.locator(':focus')).toBeVisible();

    // Test des raccourcis clavier
    await page.keyboard.press('Escape');
    // Vérifier que le modal se ferme si ouvert
  });

  test('should have proper ARIA labels', async ({ page }) => {
    await page.goto('/dashboard');

    // Vérifier les labels ARIA
    await expect(page.locator('[aria-label]')).toBeVisible();

    // Vérifier les descriptions
    await expect(page.locator('[aria-describedby]')).toBeVisible();
  });

  test('should have proper heading structure', async ({ page }) => {
    await page.goto('/dashboard');

    // Vérifier la hiérarchie des headings
    const h1Count = await page.locator('h1').count();
    expect(h1Count).toBeGreaterThan(0);

    // Vérifier l'ordre des headings
    const headings = await page.locator('h1, h2, h3, h4, h5, h6').allTextContents();
    expect(headings.length).toBeGreaterThan(0);
  });
});
```

```
});  
});
```

Tests de performance

Commandes principales :

```
# Tests de performance (recommandé)  
make test-performance  
  
# Tests de sécurité  
make test-security  
  
# Tests de base de données  
make test-database  
  
# Tests complets (tous types)  
make test-all  
  
# Alternative si make non disponible :  
node tests/performance/test-performance.js  
node tests/security/test-security.js  
node tests/database/test-database.js  
node tests/run-tests.js
```

Frontend

```
test.describe('Performance tests', () => {  
  test('should load within performance budget', async ({ page }) => {  
    const startTime = Date.now();  
  
    await page.goto('/dashboard');  
    await page.waitForLoadState('networkidle');  
  
    const loadTime = Date.now() - startTime;  
    expect(loadTime).toBeLessThan(3000); // 3 secondes max  
  });  
  
  test('should have optimized bundle size', async ({ page }) => {  
    await page.goto('/dashboard');  
  
    // Mesurer les requêtes réseau  
    const requests = await page.context().request.all();  
    const totalSize = requests.reduce((total, request) => {
```

```

    return total + (request.size() || 0);
  }, 0);

  expect(totalSize).toBeLessThan(5 * 1024 * 1024); // 5MB max
});
});

```

Backend

```

describe('API Performance', () => {
  test('should respond within acceptable time', async () => {
    const startTime = Date.now();

    await request(app)
      .get('/api/users')
      .expect(200);

    const responseTime = Date.now() - startTime;
    expect(responseTime).toBeLessThan(1000); // 1 seconde max
  });

  test('should handle concurrent requests', async () => {
    const concurrentRequests = 10;
    const requests = [];

    for (let i = 0; i < concurrentRequests; i++) {
      requests.push(
        request(app)
          .get('/api/users')
          .expect(200)
      );
    }

    const responses = await Promise.all(requests);
    expect(responses).toHaveLength(concurrentRequests);
  });
});

```

Tests de sécurité

Commandes principales :

```

# Tests de sécurité (recommandé)
make test-security

```

```
# Tests de sécurité des variables d'environnement
make test-secure-env

# Alternative si make non disponible :
node tests/security/test-security.js
node tests/security/test-secure-env-vars.js
```

API Security

```
describe('Security Tests', () => {
  test('should prevent SQL injection', async () => {
    const maliciousInput = ''; DROP TABLE users; --";

    await request(app)
      .get(`/api/users?search=${encodeURIComponent(maliciousInput)}`)
      .expect(200); // Ne devrait pas planter

    // Vérifier que la requête n'a pas causé de damage
  });

  test('should prevent XSS attacks', async () => {
    const xssPayload = '<script>alert("xss")</script>';

    const response = await request(app)
      .post('/api/users')
      .send({ name: xssPayload })
      .expect(400); // Devrait être rejeté

    expect(response.body.error).toContain('invalid');
  });

  test('should enforce rate limiting', async () => {
    const requests = [];

    // Envoyer plusieurs requêtes rapides
    for (let i = 0; i < 100; i++) {
      requests.push(
        request(app)
          .get('/api/users')
          .expect((res) => {
            // La 100ème requête devrait être rate limited
            if (i >= 99) {
              expect(res.status).toBe(429);
            }
          })
      );
    }
  });
});
```

```

    await Promise.all(requests);
  });
});

```

Authentication & Authorization

```

describe('Auth Security', () => {
  test('should require authentication for protected routes', async () => {
    await request(app)
      .get('/api/admin/users')
      .expect(401);
  });

  test('should validate JWT tokens', async () => {
    const invalidToken = 'invalid.jwt.token';

    await request(app)
      .get('/api/users/profile')
      .set('Authorization', `Bearer ${invalidToken}`)
      .expect(401);
  });

  test('should enforce role-based access', async () => {
    const userToken = 'valid.user.token';

    await request(app)
      .delete('/api/admin/users/123')
      .set('Authorization', `Bearer ${userToken}`)
      .expect(403);
  });
});

```

Tests de base de données

Commandes principales :

```

# Tests de base de données (recommandé)
make test-database

# Tests des images Docker
make test-docker-images

# Tests du système complet

```

```
make test-system-verify

# Alternative si make non disponible :
node tests/database/test-database.js
node tests/docker/test-docker-images.js
node tests/system/verify-test-system.js
```

Setup

```
// tests/setup.js
const { setupDatabase, teardownDatabase } = require('../utils/test-db');

beforeAll(async () => {
  await setupDatabase();
});

afterAll(async () => {
  await teardownDatabase();
});

afterEach(async () => {
  await clearDatabase();
});
```

Tests de migration

```
describe('Database Migrations', () => {
  test('should create required tables', async () => {
    const tables = await db.query(`
      SELECT table_name
      FROM information_schema.tables
      WHERE table_schema = 'public'
    `);

    expect(tables.rows).toEqual(
      expect.arrayContaining([
        expect.objectContaining({ table_name: 'users' }),
        expect.objectContaining({ table_name: 'applications' }),
        expect.objectContaining({ table_name: 'companies' })
      ])
    );
  });

  test('should handle data migration correctly', async () => {
    // Insérer des données de test
```

```

await db.query(`
  INSERT INTO users (email, name)
  VALUES ('test@example.com', 'Test User')
`);

// Vérifier la migration
const result = await db.query('SELECT * FROM users WHERE email = $1', ['test@e:
expect(result.rows).toHaveLength(1);
expect(result.rows[0]).toHaveProperty('id');
});
});

```

Tests de charge

Commandes principales :

```

# Tests de performance et charge (recommandé)
make test-performance

# Tests de charge backend
node tests/performance/test-performance.js

# Tests avec autocannon (si disponible)
npx autocannon http://localhost:3000/api/users -c 10 -d 10

# Alternative avec curl :
for i in {1..100}; do curl -s http://localhost:3000/api/users; done

```

Backend

```

const autocannon = require('autocannon');

describe('Load Tests', () => {
  test('should handle 1000 requests per second', async () => {
    const result = await autocannon({
      url: 'http://localhost:3000/api/users',
      connections: 10,
      duration: 10,
      requests: [
        {
          method: 'GET',
        }
      ]
    });
  });
});

```



```
    expect(result.requests.average).toBeGreaterThan(1000);  
    expect(result.errors).toBe(0);  
  });  
});
```

Frontend

```
test.describe('Frontend Load Tests', () => {  
  test('should handle rapid navigation', async ({ page }) => {  
    await page.goto('/dashboard');  
  
    const startTime = Date.now();  
  
    // Navigation rapide entre les pages  
    for (let i = 0; i < 10; i++) {  
      await page.goto('/applications');  
      await page.goto('/companies');  
      await page.goto('/dashboard');  
    }  
  
    const totalTime = Date.now() - startTime;  
    expect(totalTime).toBeLessThan(10000); // 10 secondes max  
  });  
});
```

Configuration CI/CD

Commandes principales :

```
# Tests complets via Makefile (recommandé)  
make test-all  
  
# Tests rapides (sans E2E)  
make test-quick  
  
# Tests avec rapport  
make test-report  
  
# Tests avec coverage  
make test-coverage  
  
# Alternative si make non disponible :  
./scripts/testing/run-tests.sh --all
```

```
node tests/run-tests.js --no-e2e
node tests/run-tests.js --report
cd tests && npm run test:coverage
```

GitHub Actions

```
name: Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run tests via Makefile
        run: make test-all

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          file: ./coverage/lcov.info
```

Makefile

```
.PHONY: test test-unit test-integration test-e2e test-all test-report test-quick test

# Tests complets (tous types)
test-all: ## Tests complets (tous types)
    ./scripts/testing/run-tests.sh --all

# Tests rapides (sans E2E)
test-quick: ## Tests rapides (sans E2E)
    node tests/run-tests.js --no-e2e

# Tests avec rapport
```

test-report: ## Tests avec génération de rapport

```
node tests/run-tests.js --report
```

```
# Tests avec coverage
```

test-coverage: ## Tests avec coverage

```
cd tests && npm run test:coverage
```

```
# Tests unitaires
```

test-unit: ## Tests unitaires

```
npm run test:unit
```

```
# Tests d'intégration
```

test-integration: ## Tests d'intégration

```
npm run test:integration
```

```
# Tests E2E
```

test-e2e: ## Tests E2E

```
npm run test:e2e
```

Métriques et rapports

Commandes principales :

```
# Tests avec coverage (recommandé)
```

```
make test-coverage
```

```
# Tests avec rapport
```

```
make test-report
```

```
# Alternative si make non disponible :
```

```
cd tests && npm run test:coverage
```

```
node tests/run-tests.js --report
```

Coverage

Seuil minimum :

- Branches: 70%
- Functions: 70%
- Lines: 70%
- Statements: 70%

Génération :

```
# Via Makefile (recommandé)
make test-coverage # Génère coverage/lcov-report/index.html

# Alternative directe :
cd tests && npm run test:coverage
```

Rapports Playwright

Commandes principales :

```
# Rapport HTML (recommandé)
make test-e2e-report

# Rapport JSON
npx playwright test --reporter=json

# Alternative via Makefile :
make test-e2e
npx playwright show-report
```

Tests d'accessibilité

Commandes principales :

```
# Tests d'accessibilité (recommandé)
make test-a11y

# Rapport axe-core
npm run test:a11y:axe

# Tests d'accessibilité complets
npm run test:a11y:all

# Score Lighthouse
npx lighthouse http://localhost:3000 --output=json --output-path=lighthouse-report
```

Bonnes pratiques

Écriture des tests

1. Noms descriptifs

```
// ❌  
test('should work', async () => { ... });  
  
// ✅  
test('should authenticate user with valid credentials', async () => { ... });
```

2. Structure AAA (Arrange, Act, Assert)

```
test('should create user', async () => {  
  // Arrange  
  const userData = { email: 'test@example.com', password: 'password123' };  
  
  // Act  
  const response = await request(app)  
    .post('/api/users')  
    .send(userData);  
  
  // Assert  
  expect(response.status).toBe(201);  
  expect(response.body).toHaveProperty('id');  
});
```

3. Tests indépendants

```
// ❌ Tests qui dépendent les uns des autres  
test('should create user', async () => { /* ... */ });  
test('should use created user', async () => { /* ... */ });  
  
// ✅ Tests indépendants  
test('should create user', async () => {  
  const userData = { email: 'test@example.com', password: 'password123' };  
  const response = await createUser(userData);  
  expect(response).toHaveProperty('id');  
});
```

Coverage

1. Tester les cas d'erreur

```
test('should handle invalid input', async () => {
  const response = await request(app)
    .post('/api/users')
    .send({ invalid: 'data' })
    .expect(400);

  expect(response.body.error).toContain('validation');
});
```

2. Tests de performance

```
test('should respond within acceptable time', async () => {
  const startTime = Date.now();

  await request(app).get('/api/users');

  const responseTime = Date.now() - startTime;
  expect(responseTime).toBeLessThan(1000);
});
```

3. Tests de sécurité

```
test('should sanitize user input', async () => {
  const maliciousInput = '<script>alert("xss")</script>';

  const response = await request(app)
    .post('/api/users')
    .send({ name: maliciousInput });

  expect(response.body.name).not.toContain('<script>');
});
```

Debugging

Commandes principales :

```
# Logs système complet (recommandé)
make logs

# Logs d'un service spécifique
make logs-service SERVICE=api-gateway
```

```
# Diagnostic complet
make diagnostic

# Status des services
make status

# Health check
make health

# Alternative si make non disponible :
npm run test 2>&1 | tee test.log
DEBUG=* npm run test
npx playwright test --debug
```

Logs de debug

Commandes Make (recommandées) :

```
# Logs en temps réel de tous les services
make logs

# Logs d'un service spécifique
make logs-service SERVICE=nom-du-service

# Diagnostic avec logs détaillés
make diag-services

# Health check avec logs d'erreur
make health
```

Commandes npm alternatives :

```
# Jest en mode verbose
DEBUG=* npm run test

# Playwright en mode debug
npx playwright test --debug

# Tests avec logs détaillés
npm run test 2>&1 | tee test.log
```

Breakpoints

```
// Backend
test('should handle request', async () => {
  debugger; // Point d'arrêt

  const response = await request(app)
    .get('/api/users')
    .expect(200);

  expect(response.body).toHaveProperty('data');
});
```

Screenshots et vidéos

```
// Playwright
test('should capture screenshot on failure', async ({ page }) => {
  await page.goto('/dashboard');

  // Capture d'écran en cas d'échec
  await page.screenshot({ path: 'debug-screenshot.png', fullPage: true });
});
```

CI/CD Integration

Commandes principales :

```
# Tests CI/CD (recommandé)
make test-all

# Tests avec coverage pour CI
make test-coverage

# Alternative si make non disponible :
./scripts/testing/run-tests.sh --all
cd tests && npm run test:coverage
```

Exécution automatique

```
# .github/workflows/tests.yml
name: Tests
```



```

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run tests via Makefile (recommandé)
        run: make test-all

      - name: Upload coverage
        uses: codecov/codecov-action@v3

```

Notifications

```

- name: Notify on failure
  if: failure()
  uses: 8398a7/action-slack@v3
  with:
    status: failure
    text: 'Tests failed! Check the logs.'
  env:
    SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK }

```

Guide rapide : Commandes par type de test

Tests quotidiens (Développement)

```

# Tests rapides pendant le développement
make test-quick                # Tests sans E2E
make test-backend-only         # Backend uniquement
make test-frontend-only        # Frontend uniquement

```

```
# Tests spécifiques
make test-unit           # Tests unitaires
make test-integration    # Tests d'intégration
make test-security       # Tests de sécurité
make test-performance    # Tests de performance

# Avec logs en temps réel
make logs                # Tous les logs
make status              # Status des services
make health              # Health check
```

Tests complets (CI/CD)

```
# Pipeline complète
make test-all            # Tous les tests
make test-coverage       # Tests avec coverage
make test-report         # Tests avec rapport

# Tests avancés
make test-e2e            # Tests E2E
make test-a11y           # Tests d'accessibilité
make test-database       # Tests DB
make test-docker-images  # Tests Docker
```

Debugging et diagnostic

```
# Logs et diagnostic
make diagnostic           # Diagnostic complet
make diag-services       # Services avec logs
make logs-service SERVICE=nom # Logs d'un service

# Status et health
make status              # Status des services
make health              # Health check complet
make ps                  # Liste des conteneurs
```

Alternatives npm (si make non disponible)

```
# Tests principaux
npm run test:*           # Tests spécifiques
node tests/run-tests.js  # Script principal
```

```
node tests/performance/*.js      # Tests performance
node tests/security/*.js         # Tests sécurité

# Frontend
npm run test:e2e                 # Tests E2E
npm run test:a11y                # Accessibilité
npm run test:ci                  # Tests avec coverage

# Debugging
npm run test 2>&1 | tee test.log  # Logs vers fichier
DEBUG=* npm run test             # Mode verbose
```

Stratégie de test recommandée

1. Développement quotidien

```
make test-quick # Tests rapides pendant le développement
```

2. Avant commit

```
make test-all  # Tests complets avant commit
make status     # Vérifier que tout fonctionne
```

3. Intégration continue

```
make test-coverage # CI/CD avec rapports de coverage
```

4. Debugging

```
make diagnostic # Problèmes complexes
make logs       # Logs en temps réel
```

Ressources

- [Jest Documentation](#)
 - [Playwright Documentation](#)
 - [React Testing Library](#)
 - [Supertest Documentation](#)
 - [axe-core Documentation](#)
 - [WCAG Guidelines](#)
 - [Makefile Guide](#)
 - [CI/CD Pipeline](#)
-

💡 **Astuce** : Utilisez `make help` pour voir toutes les commandes disponibles et `make help-test` pour l'aide spécifique aux tests.

Ce guide sera mis à jour régulièrement pour refléter les meilleures pratiques et les nouveaux outils.